

ARTISAN AI

Technical & Product Architecture Document

AI-Driven Market Linkage and Smart Cataloging Mobile Application for Marginalized Artisans

Smart India Hackathon 2026 | Problem Statement ID: 26090 | Theme: Heritage & Culture | PS Category: Software

Version 1.0 — Production-Oriented Hackathon Architecture

1. Executive Summary

Artisan AI is a voice-first AI business platform designed for marginalized artisans and weavers. It reduces the digital barrier to commerce by allowing an artisan to create a professional product listing using a product photograph and a multilingual voice description, then assists with explainable pricing, market discovery, buyer linkage and business decisions.

The product is designed as a two-sided commerce experience. Users can switch between SELL and BUY modes within the same account. SELL mode focuses on products, demand signals, pricing, orders and business insights. BUY mode focuses on discovery, personalized recommendations, trending products, suppliers, enquiries and orders.

The central intelligence layer is the Market Intelligence Engine. It collects buyer-side signals such as searches, views, saves, enquiries and orders; converts them into demand and trend signals; and exposes actionable intelligence to sellers. These signals can also inform explainable price recommendations and buyer recommendations.

The architecture remains aligned with the submitted concept: AI image enhancement, multilingual voice-to-catalog, explainable pricing, Business Copilot, market linkage and offline queueing.

2. Problem Statement

Problem Statement ID: 26090

Title: AI-Driven Market Linkage and Smart Cataloging Mobile Application for Marginalized Artisans

The target audience includes marginalized micro-entrepreneurs, artisans and weavers in rural and semi-urban India. The core challenge is not simply lack of a marketplace; it is the combined difficulty of digital catalog creation, language and typing barriers, product presentation, pricing decisions and access to B2C/B2B buyers.

2.1 Core problems

- Low digital literacy makes conventional seller onboarding and product listing difficult.
- Artisans may struggle to produce professional product photographs and descriptions.
- Typing-heavy workflows create language and usability barriers.
- Pricing can be difficult when material, labour, craft value and market conditions vary.
- Artisans may have limited visibility into buyer demand and suitable B2C/B2B opportunities.
- Intermittent rural connectivity can interrupt uploads and workflows.
- AI-generated content must remain editable and subject to artisan approval.

3. Product Vision

From a simple product photo and voice description to a complete, explainable and market-aware business workflow.

PHOTO + VOICE → SMART CATALOG → FAIR/EXPLAINABLE PRICE → MARKET LINKAGE → BUSINESS INTELLIGENCE

3.1 Product promise

Artisan AI should make professional digital commerce possible with minimal typing while continuously helping the artisan understand what is happening in the market and what action may be useful next.

4. Core Innovation

The innovation is not a single AI model. It is the closed-loop architecture that connects cataloging, marketplace behavior, seller intelligence, pricing and recommendations.

BUYER ACTIVITY → MARKET INTELLIGENCE → SELLER INSIGHTS → PRICE/SUPPLY ACTION → BETTER BUYER DISCOVERY

- Voice-first, multilingual product creation.
- AI-assisted professional product imagery and catalog generation.
- Explainable price recommendations rather than an unexplained single number.
- SELL ↔ BUY mode switching in one application.
- A shared Market Intelligence Engine that converts buyer behavior into seller-facing demand signals.
- Personalized buyer recommendations plus marketplace-wide trending signals.
- Seller Product Intelligence pages with full performance, demand, inventory and pricing context.
- Offline-first queueing and resumable synchronization for intermittent connectivity.
- Human approval before publishing AI-generated catalog or pricing decisions.

5. User Roles & Experience

5.1 SELL mode

- Seller dashboard
- Add product using photo and voice
- AI-generated catalog
- Product details and performance
- Demand and trend insights
- Explainable dynamic price recommendation
- Orders and enquiries
- Buyer opportunities
- Business Copilot

5.2 BUY mode

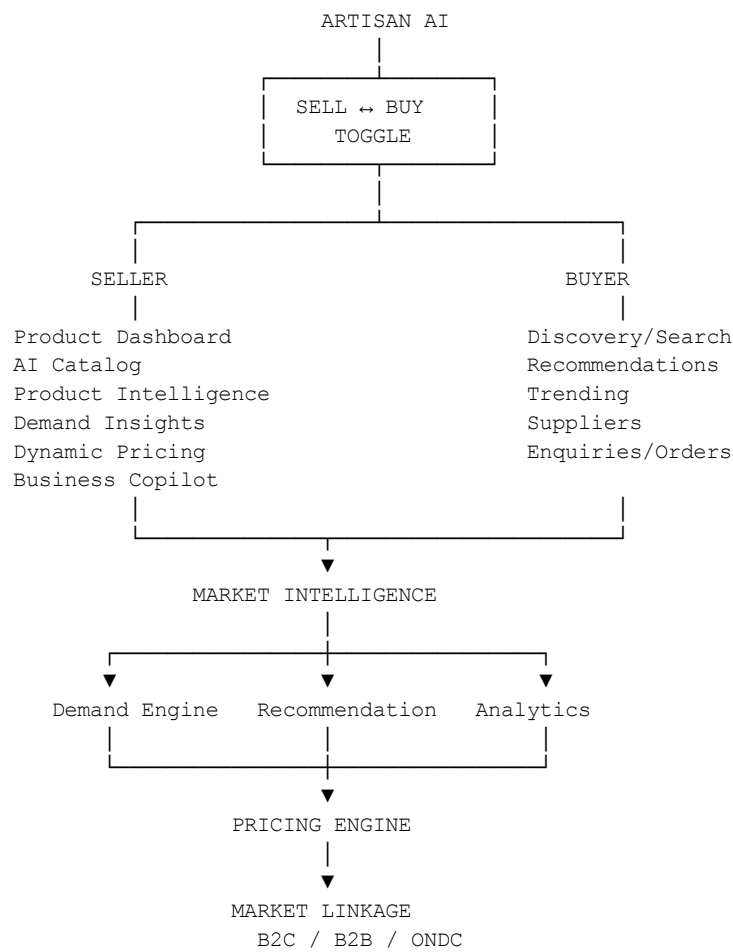
- Search and discovery
- Personalized recommendations
- Most searched / trending products
- Supplier and artisan discovery
- B2B enquiries and bulk requirements
- Orders and saved products

5.3 Mode switch

[🧑 SELL] ↔ [🏢 BUY]

A single account may maintain both seller and buyer capabilities. The active mode changes the dashboard and primary actions without requiring a second account.

6. Functional Architecture



7. AI Product Creation & Smart Catalog

7.1 Input

PHOTO + VOICE + OPTIONAL COST INPUT

7.2 Processing pipeline

1. Capture or select product image.
2. Compress/store the media locally when required.
3. Run background removal, lighting enhancement and upscaling.
4. Transcribe voice input using speech recognition.
5. Translate/normalize multilingual input when required.
6. Use vision and language intelligence to understand product attributes.
7. Generate title, description, materials, category, tags and craft story.
8. Present the generated catalog for seller editing.
9. Save a versioned catalog and publish only after seller approval.

7.3 Image enhancement

Original Photo → Background Removal → Lighting Enhancement → Upscaling → Product-Ready Image

7.4 Human-in-the-loop

AI-generated catalog content is a draft. The artisan remains in control through preview, edit and approve actions.

8. Seller Product Intelligence Dashboard

Clicking a product must open a complete intelligence page rather than only a basic inventory record.

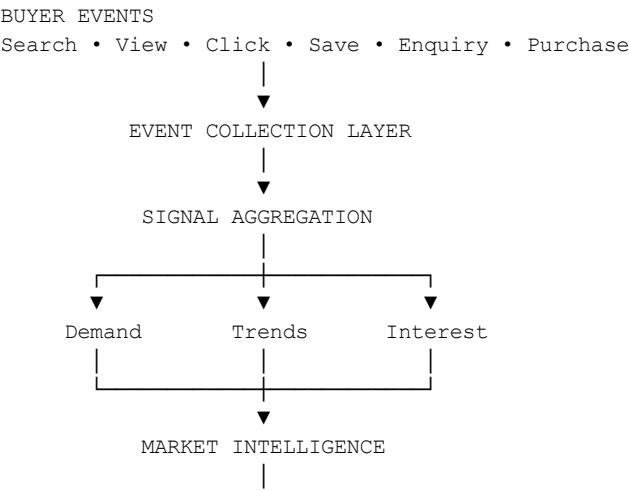
PRODUCT INTELLIGENCE
Product image(s)
Title / description / craft story
Materials / category / tags
PERFORMANCE
Views Saves Orders Enquiries Conversion
DEMAND
Search interest Demand level Trend direction
INVENTORY
Available stock Units sold Sales velocity
PRICE
Current price Recommended price Price rationale
AI ACTION
Recommended next step
[Accept] [Edit] [Keep Current]

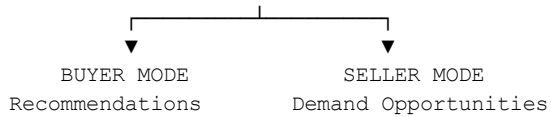
8.1 Example insight

“Demand is increasing. Search interest is up 32%, comparable products are within the ₹1,150–₹1,400 range, and your current inventory is low. Consider preparing additional units and review the suggested price.”

9. Market Intelligence Engine

This is the central feedback layer connecting buyer activity with seller decisions.





9.1 Signals

- Search frequency
- Product/category views
- Click-through behavior
- Saves/wishlist actions
- Enquiries
- Orders and sales velocity
- Inventory availability
- Category-level demand
- Buyer interest profiles
- Available market-price signals

10. Buyer Recommendation Engine

10.1 Personalized recommendations

Recommendations use a buyer's observed interests and interactions to surface relevant products.

SEARCH + VIEW + SAVE + ENQUIRY + PURCHASE → INTEREST PROFILE → PERSONALIZED RECOMMENDATIONS

10.2 Trending / most searched

Trending is different from personalization. It is an aggregate marketplace signal and can be displayed to all relevant buyers.

Marketplace Searches + Engagement + Orders → TRENDING PRODUCTS

10.3 Cold-start strategy

- Use category-level popularity and trending signals for new buyers.
- Use onboarding interests when available.
- Use product metadata and semantic similarity before sufficient behavioral history exists.

11. Seller Demand Intelligence

The same market signals used to improve buyer discovery become actionable signals for sellers.

MARKET DEMAND
Kalamkari ↑32%
Wooden Toys ↑24%
Pottery ↑18%

SELLER VIEW
"High demand detected"
"Your Kalamkari inventory is low"
"Consider reviewing price and stock"

The system should distinguish market-level demand from a seller's own product performance. A high-demand category does not automatically mean every product should be repriced or restocked.

12. Explainable Dynamic Pricing

Dynamic pricing is implemented as a recommendation engine. The platform should not silently change a seller's price.



12.1 Profit-oriented logic

A conceptual objective is to maximize expected profit rather than simply maximize sales volume:

$$\text{Expected Profit} \approx (\text{Recommended Price} - \text{Cost}) \times \text{Expected Demand}$$

The production system should not claim statistically reliable demand elasticity until sufficient real transaction data exists. In the hackathon MVP, seeded or simulated market signals can demonstrate the mechanism, while the product clearly labels them as demo data.

12.2 Price explanation

- Current price
- Recommended price/range
- Cost basis
- Demand trend
- Comparable market range
- Inventory condition
- Expected margin
- Seller approval controls

13. AI Business Copilot

The Business Copilot converts raw analytics into concise, actionable recommendations.

 AI BUSINESS COPILOT

Demand ↑ 32%
Stock: 6 units
Buyer enquiries: 7
Comparable price: ₹1,150-₹1,400

NEXT BEST ACTION

- Review price: ₹1,199 → ₹1,249
- Consider producing 10-15 more units
- Review 3 relevant buyer enquiries

[View Opportunity]

The Copilot should explain the evidence behind recommendations and avoid pretending that uncertain predictions are guaranteed outcomes.

14. Marketplace & Market Linkage

14.1 B2C

Individual buyers discover products, receive personalized recommendations, save items, enquire and order.

14.2 B2B

Businesses can discover artisans/products, make enquiries, request quantities and pursue bulk orders.

14.3 ONDC strategy

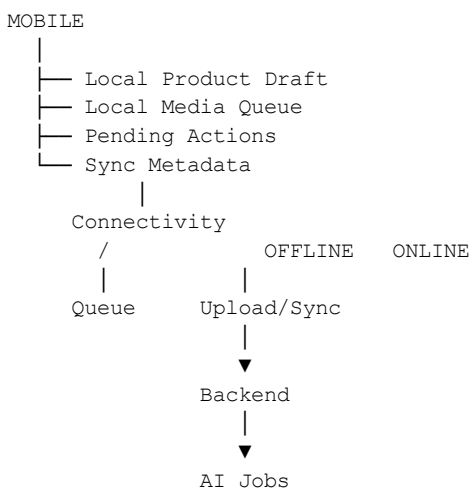
ONDC should be treated as a market-linkage/integration layer. The architecture should isolate ONDC-specific integration behind an adapter so the core product does not depend on a single external protocol.

Artisan AI Marketplace Core → ONDC Adapter → ONDC Network

The prototype should only claim the ONDC capabilities that are actually implemented and tested.

15. Offline-First Architecture

The submitted concept includes offline queueing for intermittent connectivity. The MVP should support local drafts and queued uploads/actions rather than attempting to run every AI model offline.

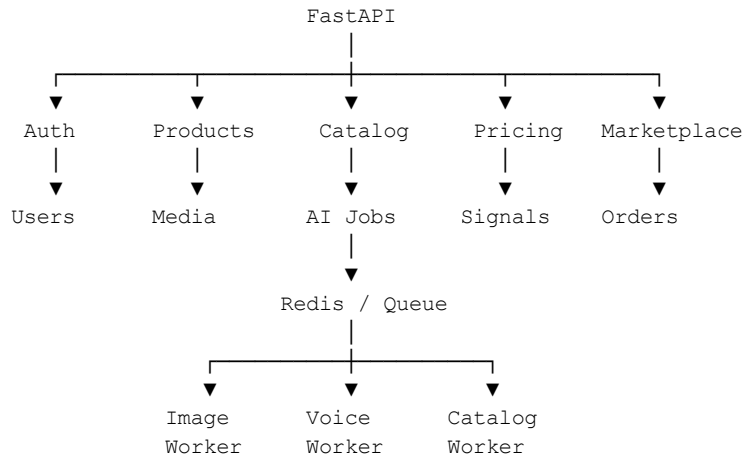


15.1 Sync requirements

- Unique operation/job IDs for idempotency.
- Retry with backoff.

- Resumable uploads for large media.
- Conflict handling using version numbers/timestamps.
- Visible sync state: Pending, Syncing, Synced, Failed.
- No data loss when connectivity changes.

16. Backend Architecture



16.1 Recommended modular services

- Authentication & authorization
- User/profile service
- Product/catalog service
- Media service
- AI orchestration service
- Pricing service
- Market intelligence service
- Recommendation service
- Order/enquiry service
- Notification service
- Marketplace/ONDC adapter
- Sync/job service

Use a modular monolith for the hackathon. Modules can be separated into services later if scale requires it.

17. Data Architecture

users
profiles
products
product_media
catalog_versions

buyer_profiles
buyer_interests
events

demand_signals

market_trends
recommendations

pricing_reports
price_history

orders
enquiries
buyer_requests

ai_jobs
sync_jobs
notifications
audit_logs
marketplace_listings

17.1 Important entities

- User: identity, authentication and capabilities.
- Profile: seller/buyer business information.
- Product: canonical product data and status.
- CatalogVersion: AI-generated and seller-edited versions.
- Event: behavioral activity used for intelligence.
- DemandSignal: aggregated demand/trend metrics.
- PricingReport: explainable pricing recommendation.
- Order/Enquiry: commerce transactions and leads.
- AIJob/SyncJob: asynchronous and offline workflow state.
- AuditLog: important user and system actions.

18. API Architecture

POST /auth/login
GET /me
PATCH /me/mode

POST /products
POST /products/{id}/media
POST /products/{id}/ai-process
GET /products
GET /products/{id}
PATCH /products/{id}
POST /products/{id}/approve

GET /products/{id}/analytics
GET /products/{id}/demand
GET /products/{id}/price-recommendation
POST /products/{id}/price-approve

GET /market/trending
GET /market/demand
GET /recommendations
GET /seller/opportunities

POST /enquiries
GET /enquiries
POST /orders

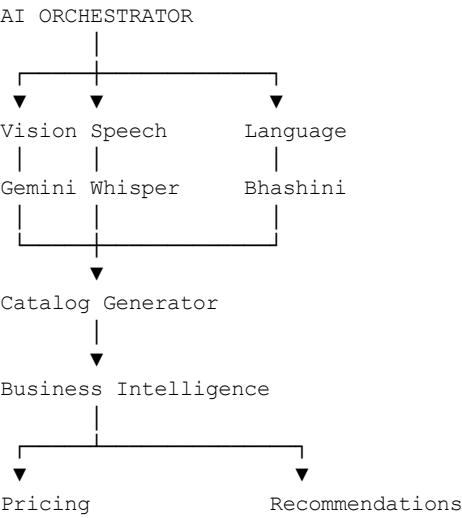
POST /sync/jobs

```
GET    /sync/status/{id}

GET    /health
GET    /ready
```

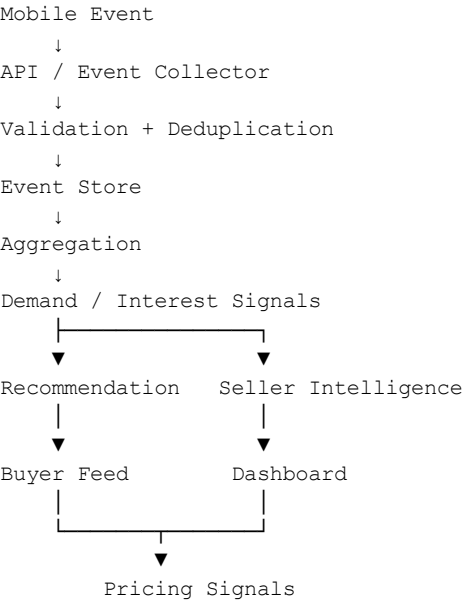
API contracts should be versioned and validated. Long-running AI work should return a job identifier rather than blocking a mobile request.

19. AI/ML Architecture



The submitted technology choices include RMBG-1.4, Whisper, Real-ESRGAN, Gemini Vision and Bhashini APIs. 穀filecite學turn2file0學L82-L91條

20. Event & Analytics Pipeline



20.1 Important event properties

- event_id

- user_id or anonymous session ID where appropriate
- product_id/category_id
- event_type
- timestamp
- source/screen
- device/network metadata only where necessary
- schema version

21. Security & Trust

- JWT/OAuth-based authentication.
- Role-based authorization for SELL and BUY capabilities.
- Server-side validation for every sensitive operation.
- Secure file upload with MIME/type/size checks.
- Signed or controlled media URLs.
- Secrets stored outside source code.
- Rate limiting on public and expensive AI endpoints.
- Audit logs for publishing, price approval and account-sensitive changes.
- AI-generated content is clearly editable and requires human approval for publishing.
- Minimize collection of personal data and protect buyer/seller information.

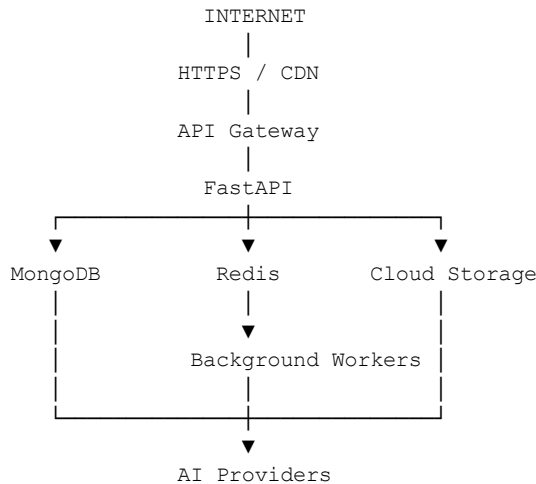
22. Reliability & Failure Handling

- AI provider timeout → retry or mark job for retry.
- Speech recognition uncertainty → show transcript for correction.
- Translation failure → preserve original text and allow manual editing.
- Image processing failure → retain original image and retry enhancement.
- Market signal unavailable → fall back to seller cost inputs and transparent baseline pricing.
- Recommendation cold start → use category/trending/product metadata.
- Network loss → save locally and queue synchronization.
- Duplicate requests → idempotency keys/job IDs.
- Partial processing → maintain job state so the pipeline can resume.

23. Observability

- Structured backend logs.
- Request IDs and AI job IDs.
- Latency and error metrics.
- AI provider success/failure rates.
- Queue depth and job age.
- Image/voice processing completion rates.
- Recommendation click-through and conversion metrics.
- Pricing recommendation acceptance rate.
- Sync success/failure rate.
- Health endpoint and readiness endpoint.

24. Production Deployment



24.1 Environment separation

- Development
- Staging
- Production

24.2 Deployment principles

- Containerize backend and workers.
- Use managed database/storage where practical.
- Use HTTPS everywhere.
- Separate secrets/configuration from application code.
- Use backups and retention policies.
- Use health/readiness checks.
- Keep AI providers behind an abstraction layer.

25. Scalability Strategy

- Scale FastAPI instances horizontally.
- Scale AI workers independently based on queue depth.
- Use Redis for short-lived queues/caching where appropriate.
- Use object storage rather than database blobs for large media.
- Index product, category, event and seller fields used by high-frequency queries.
- Precompute aggregate demand signals rather than recalculating them for every dashboard request.
- Cache trending results for short periods.
- Introduce specialized search infrastructure only when scale justifies it.

26. Key User Journeys

26.1 Seller journey

Onboard → SELL → Add Product → Photo + Voice → AI Catalog → Edit → AI Price → Approve → Publish → Monitor → Act

26.2 Buyer journey

BUY → Choose interests → Search/Browse → Personalized Recommendations → Trending → Product Details → Enquiry/Order

26.3 Closed-loop journey

Buyer Searches → Signal Increases → Seller Sees Demand → Reviews Stock/Price → Better Listing/Supply → Buyer Gets Better Discovery

27. Hackathon MVP Scope

Must work end-to-end

10. SELL ↔ BUY toggle.
11. Seller onboarding.
12. Photo upload/camera flow.
13. Voice input with at least one Indian language path.
14. AI catalog generation.
15. Image enhancement demonstration.
16. Seller product dashboard.
17. Full product intelligence page.
18. Seeded/controlled demand signals.
19. Buyer personalized recommendations.
20. Trending/most-searched products.
21. Explainable price recommendation.
22. Seller approval flow.
23. Basic B2C/B2B enquiry flow.
24. Offline draft/queue demonstration.
25. Production-style authentication and validation.

27.1 Do not overbuild

- Do not build dozens of microservices.
- Do not train a complex pricing model without data.
- Do not claim live ONDC functionality unless actually integrated and tested.
- Do not attempt full offline inference for every AI model.
- Do not add unrelated features merely to increase feature count.

28. Winning Hackathon Demo Flow

26. Open the app in SELL mode.
27. Capture a handmade product image.
28. Speak a short description in a supported Indian language.
29. Show image enhancement and AI-generated catalog.
30. Show seller editing and approval.
31. Open the full Product Intelligence page.
32. Show demand trend and buyer interest.
33. Show explainable price recommendation.
34. Switch to BUY mode.

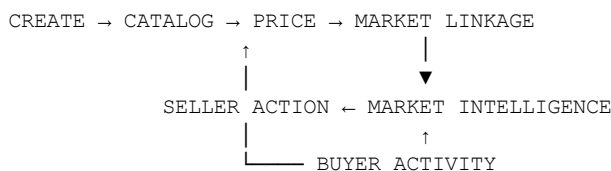
35. Search/view/save a product category.
36. Show personalized recommendations and trending products.
37. Switch back to SELL mode.
38. Show how the buyer activity has generated a seller-facing demand insight.
39. Show the AI Business Copilot recommendation.
40. Demonstrate offline draft/queued sync.
41. Finish with B2C/B2B market linkage and the ONDC adapter architecture.

28.1 Judge-facing story

“An artisan should not need to understand e-commerce software. They give us a photo and their voice. Artisan AI turns that into a professional listing, explains a fair price, connects the product to buyers, and continuously turns marketplace behavior into actionable business intelligence.”

29. Competitive Differentiation

The project should avoid claiming that individual features such as marketplaces, AI catalog generation, recommendations or dynamic pricing are globally unique. The defensible differentiation is the way these capabilities are connected into a single artisan-first workflow.



- Designed around the artisan's minimum-input workflow.
- Buyer behavior becomes useful seller intelligence.
- Product-level analytics combine performance, demand, inventory and pricing context.
- Dynamic pricing is explainable and seller-controlled.
- One account supports both SELL and BUY modes.
- The system is designed for intermittent connectivity and multilingual input.

30. Impact

Social

- Lower digital barriers through voice-first interaction.
- Greater independence in managing product information.
- Reduced dependence on typing-heavy workflows.

Economic

- Professional product presentation.
- More informed pricing decisions.
- Improved buyer discovery.
- Better B2C and B2B market access.

Cultural

- Preserve artisan stories and craft identity in digital catalogs.

- Increase digital visibility of traditional crafts.

These impacts are consistent with the submitted solution's stated social, economic and cultural benefits.

穀filecite學turn2file0學L142-L164傢

31. Implementation Roadmap

Phase 1 — Core MVP

- React Native/Expo app shell
- SELL/BUY mode switch
- Authentication
- Product creation
- AI catalog pipeline
- Product dashboard

Phase 2 — Intelligence

- Event collection
- Trending engine
- Buyer recommendations
- Seller demand signals
- Product analytics

Phase 3 — Commerce

- Explainable pricing
- Orders/enquiries
- B2B workflows
- Marketplace adapter

Phase 4 — Production Hardening

- Offline/resumable sync
- Observability
- Security hardening
- Performance optimization
- Automated testing
- Staging/production deployment

32. Testing Strategy

- Unit tests for pricing, permissions, event aggregation and validation.
- Integration tests for AI job lifecycle and database operations.
- API contract tests.
- Mobile offline/online transition tests.
- Media upload/resume tests.
- Recommendation cold-start tests.
- Pricing boundary tests to protect minimum seller margin.
- Security tests for authorization and upload handling.

- End-to-end demo test covering SELL → BUY → market feedback loop.

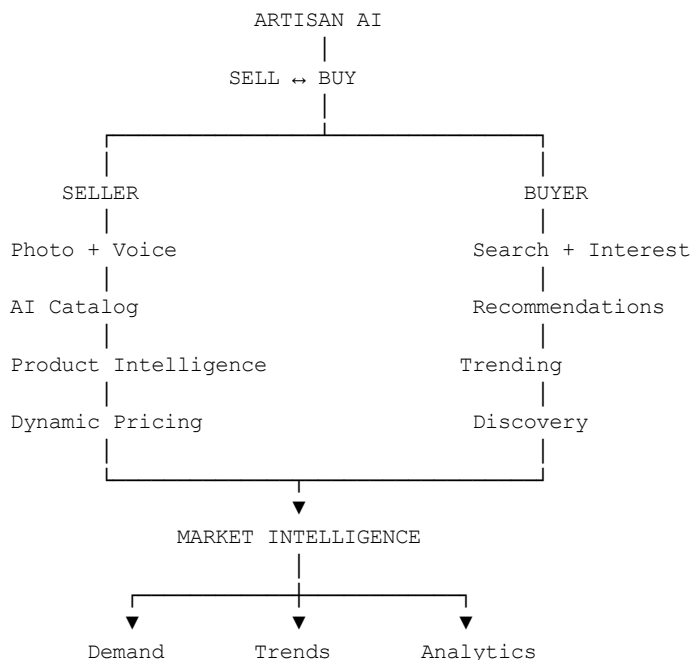
33. Product KPIs

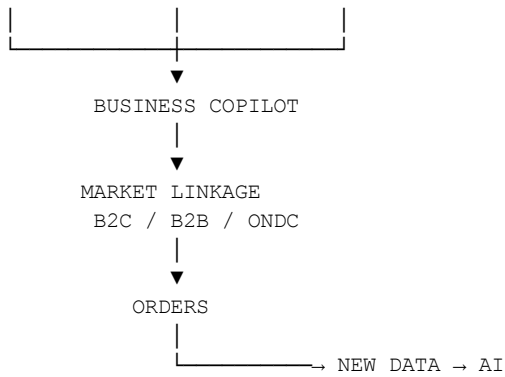
- Time from product capture to approved catalog.
- Catalog completion/approval rate.
- AI transcription correction rate.
- Listing publish rate.
- Buyer recommendation click-through rate.
- Search-to-enquiry conversion.
- Enquiry-to-order conversion.
- Pricing recommendation acceptance rate.
- Seller repeat usage.
- Offline sync success rate.
- B2B enquiry volume.
- Buyer and seller retention.

34. Risks & Mitigations

- Regional accents/noise → confidence-aware transcription and correction.
- Incomplete market data → transparent signal labels and fallback pricing logic.
- AI hallucinated product details → seller edit/approve workflow.
- Demand spikes caused by small samples → minimum-volume thresholds and smoothing.
- Price volatility → bounded recommendations and minimum-margin rules.
- Connectivity → local queue and resumable sync.
- External marketplace/API changes → adapter abstraction.

35. Final Architecture Summary





Final positioning: Artisan AI is a voice-first AI business platform for marginalized artisans that combines smart cataloging, explainable pricing, buyer discovery and market intelligence in one continuous commerce workflow.

Source alignment note: the original SIH submission specifies the problem statement, core solution, technical stack, methodology, feasibility considerations and impact. This document expands those submitted components into a production-oriented architecture and implementation plan.

filecite學turn2file0學L35-L69像